

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №6

**«Сборка многомодульных программ.
Вычисление корней уравнений и определенных
интегралов»**

Вариант 2 / 2,3 / 2

Выполнила:
студентка 104 группы
Грознецких Екатерина Дмитриевна

Преподаватель:
Гуляев Денис Анатольевич

Москва
2026

Содержание

Постановка задачи	2
Математическое обоснование	3
Результаты экспериментов	6
Структура программы и спецификация функций	7
Сборка программы (Make-файл)	9
Отладка программы, тестирование функций	10
Программа на Си и на Ассемблере	11
Анализ допущенных ошибок	12
Список литературы	13

Постановка задачи

Требуется реализовать многомодульную программу на языке Си и языке ассемблера NASM32, вычисляющую площадь плоской фигуры, ограниченной графиками трех функций. В соответствии с вариантом 2 заданы функции

$$f_1(x) = 3 \left(\frac{0.5}{x+1} + 1 \right), \quad f_2(x) = 2.5x - 9.5, \quad f_3(x) = \frac{5}{x}, \quad x > 0.$$

В программе используется тип данных `long double`. Функции f_1 , f_2 , f_3 , а также их производные реализованы в отдельном ассемблерном модуле `funcs.asm`. Остальная часть программы, включая поиск корней, вычисление интегралов, обработку ключей командной строки и вывод результатов, реализована в модуле `main.c`.

Вершины фигуры находятся как точки пересечения пар кривых. Для этого решаются уравнения

$$f_1(x) = f_3(x), \quad f_2(x) = f_3(x), \quad f_1(x) = f_2(x).$$

Поиск корней выполняется комбинированным методом: на каждом шаге используется метод хорд и метод Ньютона. Для вычисления площади применяется метод трапеций. В экспериментах используются значения точности

$$\varepsilon = 10^{-3}, \quad \varepsilon = 10^{-5}, \quad \varepsilon = 10^{-7}.$$

Отрезки, на которых применяются численные методы нахождения корней, предварительно выбираются аналитически по виду уравнений и знакам разностей функций на концах отрезков. В программе использованы отрезки $[1, 2]$, $[4, 5]$ и $[5, 6]$.

Математическое обоснование

Заданная первая функция может быть записана в более удобном виде:

$$f_1(x) = 3 \left(\frac{0.5}{x+1} + 1 \right) = 3 + \frac{1.5}{x+1}.$$

Производные функций, необходимые для метода Ньютона, равны

$$f'_1(x) = -\frac{1.5}{(x+1)^2}, \quad f'_2(x) = 2.5,$$

$$f'_3(x) = -\frac{5}{x^2}.$$

Все эти функции определены на рассматриваемых отрезках, причем для f_3 дополнительно учитывается ограничение $x > 0$.

Для выбора отрезков поиска точек пересечения были аналитически рассмотрены уравнения пересечения пар функций.

Для пересечения f_1 и f_3 :

$$3 + \frac{1.5}{x+1} = \frac{5}{x}.$$

После умножения на $x(x+1)$ получаем

$$3x^2 - 0.5x - 5 = 0,$$

или

$$6x^2 - x - 10 = 0.$$

Положительный корень равен

$$x_{13} = \frac{1 + \sqrt{241}}{12} \approx 1.3770.$$

Поэтому для поиска этого корня выбран отрезок $[1, 2]$.

Для пересечения f_2 и f_3 :

$$2.5x - 9.5 = \frac{5}{x}.$$

После умножения на x получаем

$$2.5x^2 - 9.5x - 5 = 0,$$

или

$$5x^2 - 19x - 10 = 0.$$

Положительный корень равен

$$x_{23} = \frac{19 + \sqrt{561}}{10} \approx 4.2685.$$

Поэтому выбран отрезок $[4, 5]$.

Для пересечения f_1 и f_2 :

$$3 + \frac{1.5}{x+1} = 2.5x - 9.5.$$

Отсюда

$$(2.5x - 12.5)(x + 1) - 1.5 = 0,$$

то есть

$$2.5x^2 - 10x - 14 = 0,$$

или

$$5x^2 - 20x - 28 = 0.$$

Положительный корень равен

$$x_{12} = 2 + 0.8\sqrt{15} \approx 5.0984.$$

Поэтому выбран отрезок $[5, 6]$.

Графики функций приведены на рис. 1. На рассматриваемой фигуре верхней границей является функция f_1 . На отрезке $[x_{13}, x_{23}]$ нижней границей является f_3 , а на отрезке $[x_{23}, x_{12}]$ нижней границей является f_2 . Поэтому площадь фигуры вычисляется по формуле

$$S = \int_{x_{13}}^{x_{23}} (f_1(x) - f_3(x)) dx + \int_{x_{23}}^{x_{12}} (f_1(x) - f_2(x)) dx.$$

Для метода Ньютона требуется, чтобы в окрестности корня функция была дифференцируема, а производная не обращалась в ноль. В программе метод применяется к разности двух функций, то есть к функции $F(x) = f(x) - g(x)$. Производная этой функции равна $F'(x) = f'(x) - g'(x)$. На выбранных отрезках функции непрерывны, их производные существуют, а сами отрезки содержат по одному корню. Метод хорд дополнительно использует значения функции на концах отрезка, где разность функций меняет знак.

В реализации одно значение ε используется и для поиска корней, и для остановки уточнения интеграла. Для метода трапеций число разбиений удваивается до тех пор, пока модуль разности двух последовательных приближений интеграла не станет меньше ε .

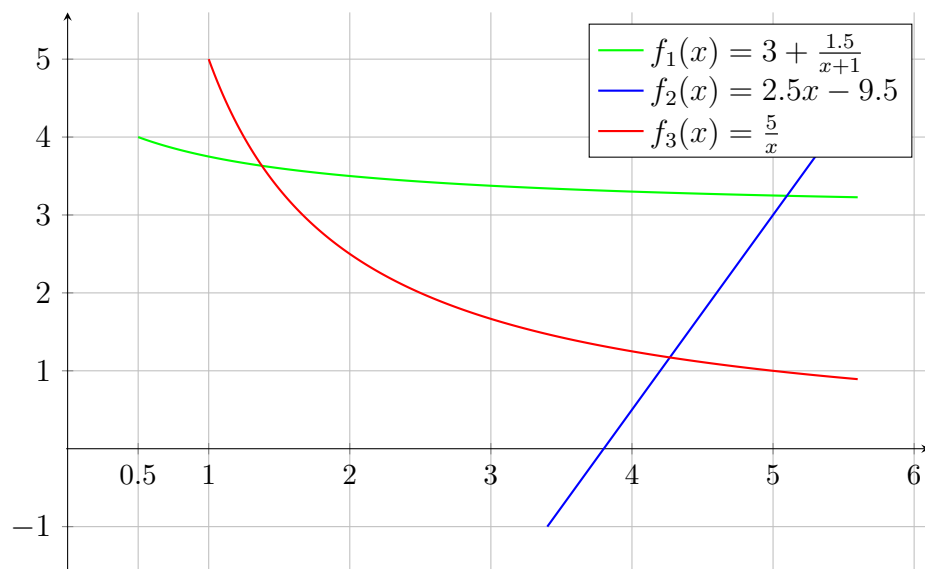


Рис. 1: Графики функций варианта 2

Результаты экспериментов

В результате работы программы были найдены точки пересечения кривых и площадь ограниченной ими фигуры. Координаты точек пересечения приведены в таблице 1. Значения y вычислены подстановкой найденных абсцисс в соответствующие функции.

Кривые	x	y
1 и 3	1.377015	3.631044
2 и 3	4.268544	1.171360
1 и 2	5.098387	3.245967

Таблица 1: Координаты точек пересечения

Результаты вычислений для различных значений точности приведены в таблице 2. В таблице также указано количество итераций, потребовавшееся функции `root` для нахождения каждой точки пересечения.

ε	x_{13}	ит.	x_{23}	ит.	x_{12}	ит.	S
10^{-3}	1.377182	6	4.268557	2	5.098387	2	5.087822
10^{-5}	1.377018	9	4.268544	3	5.098387	2	5.087770
10^{-7}	1.377015	13	4.268544	4	5.098387	2	5.087767

Таблица 2: Результаты работы программы

При уменьшении ε значения корней и площади стабилизируются. Итоговое значение площади при $\varepsilon = 10^{-7}$:

$$S \approx 5.087767.$$

На рис. 2 показана ограниченная область, площадь которой вычислялась программой.

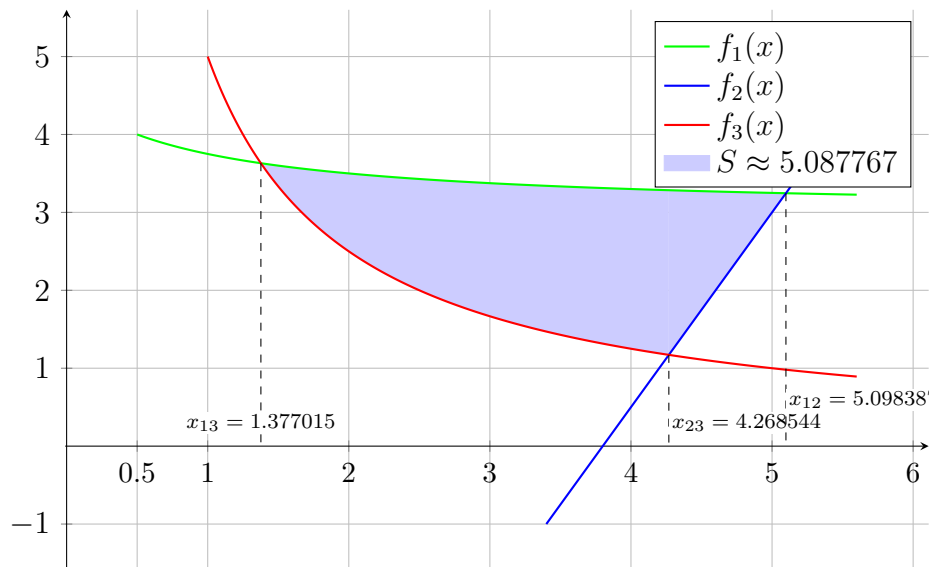


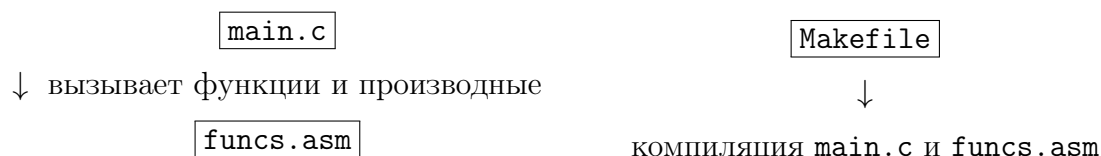
Рис. 2: Плоская фигура, ограниченная графиками заданных функций

Структура программы и спецификация функций

Программа состоит из трех файлов:

- `main.c` — основной модуль на языке Си;
- `funcs.asm` — модуль на ассемблере NASM32 с функциями и производными;
- `Makefile` — файл сборки проекта.

Общая схема взаимодействия модулей показана ниже.



Ниже приведена спецификация функций программы.

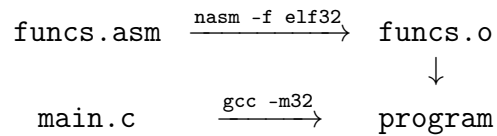
Функция	Назначение
<code>f1(long double x)</code>	Ассемблерная функция. Возвращает значение $f_1(x) = 3 + \frac{1.5}{x+1}$. Значение аргумента x передается из Си-кода, результат возвращается через регистр <code>st(0)</code> сопроцессора x87.
<code>f2(long double x)</code>	Ассемблерная функция. Возвращает значение $f_2(x) = 2.5x - 9.5$. Используется при поиске корней, вычислении интегралов и тестировании.
<code>f3(long double x)</code>	Ассемблерная функция. Возвращает значение $f_3(x) = \frac{5}{x}$. В программе вызывается только на положительных значениях x , так как по условию $x > 0$.
<code>df1(long double x)</code>	Ассемблерная функция. Возвращает производную $f'_1(x) = -\frac{1.5}{(x+1)^2}$, необходимую для метода Ньютона.
<code>df2(long double x)</code>	Ассемблерная функция. Возвращает производную $f'_2(x) = 2.5$.
<code>df3(long double x)</code>	Ассемблерная функция. Возвращает производную $f'_3(x) = -\frac{5}{x^2}$.
<code>raznost(func_t f, func_t g, long double x)</code>	Возвращает значение разности двух функций $f(x) - g(x)$. Эта функция используется для преобразования задачи пересечения двух графиков к задаче поиска корня одной функции.
<code>raznostproisv(func_t df, func_t dg, long double x)</code>	Возвращает разность производных $f'(x) - g'(x)$. Используется внутри метода Ньютона для функции $F(x) = f(x) - g(x)$.

<code>root(...)</code>	Основная функция поиска корня уравнения $f(x) = g(x)$ на отрезке $[a, b]$. На каждой итерации вычисляет приближение методом хорд и приближение методом Ньютона. Итерации продолжаются, пока расстояние между этими приближениями больше заданной точности ε . Через указатель iterations возвращается число итераций.
<code>integral(func_t f, long double a, long double b, long double eps)</code>	Вычисляет определенный интеграл функции f на отрезке $[a, b]$ методом трапеций. Число разбиений начинается с 4 и удваивается до тех пор, пока разность двух последовательных приближений не станет меньше ε .
<code>get_func(char *name)</code>	По строковому имени "f1", "f2" или "f3" возвращает указатель на соответствующую функцию. Используется для обработки ключей -test-root и -test-integral .
<code>get_dfunc(char *name)</code>	По имени функции возвращает указатель на ее производную: df1 , df2 или df3 . Используется при тестировании функции root .
<code>print_help(void)</code>	Выводит список доступных режимов запуска программы и формат аргументов командной строки.
<code>calculate(...)</code>	Выполняет основные вычисления: находит три точки пересечения, вычисляет площадь фигуры и возвращает количество итераций для каждого корня.
<code>print_result(long double eps)</code>	Вызывает calculate для указанного значения ε и выводит корни, количество итераций и площадь. Этот режим используется при запуске программы без аргументов.
<code>print_roots(void)</code>	Выводит только найденные корни для точности 10^{-7} . Соответствует ключу -roots .
<code>print_iterations(void)</code>	Выводит только количество итераций для трех корней при точности 10^{-7} . Соответствует ключу -iterations .
<code>main(int argc, char *argv[])</code>	Главная функция программы. Анализирует аргументы командной строки, выбирает режим работы, запускает основные вычисления или тестовые режимы. При запуске без аргументов выводит результаты для 10^{-3} , 10^{-5} и 10^{-7} .

Сборка программы (Make-файл)

Сборка программы выполняется с помощью `make`. Ассемблерный файл `funcs.asm` транслируется NASM в 32-битный объектный файл `funcs.o`. Затем `main.c` и `funcs.o` собираются компилятором `gcc` в исполняемый файл `program`.

Схема зависимостей:



Текст Make-файла:

```
all:
    nasm -f elf32 funcs.asm -o funcs.o
    gcc -m32 -no-pie main.c funcs.o -lm -o program

clean:
    rm -f *.o program
```

Для сборки и запуска используются команды:

```
make
./program
```

Для удаления объектных файлов и исполняемого файла используется команда:

```
make clean
```

Отладка программы, тестирование функций

Для проверки программы были реализованы ключи командной строки:

- `-help` — вывод справки;
- `-roots` — вывод найденных корней;
- `-iterations` — вывод количества итераций;
- `-test-root f1 f2 a b eps` — тестирование функции `root`;
- `-test-integral f1 a b eps` — тестирование функции `integral`.

Проверка функции `root` проводилась на уравнениях пересечения заданных функций. Эти уравнения имеют аналитически вычисляемые корни, что позволяет сравнить численный ответ с точным.

Команда	Точный корень	Численный результат	Итерации
<code>-test-root f1 f2 5 6 0.001</code>	$2 + 0.8\sqrt{15}$	5.098387	2
<code>-test-root f1 f3 1 2 0.001</code>	$\frac{1+\sqrt{241}}{12}$	1.377182	6
<code>-test-root f2 f3 4 5 0.001</code>	$\frac{19+\sqrt{561}}{10}$	4.268557	2

Таблица 4: Тестирование функции `root`

Проверка функции `integral` проводилась на тех же функциях. Для них первообразные имеют простой аналитический вид:

$$\int f_1(x) dx = 3x + 1.5 \ln |x + 1| + C,$$

$$\int f_2(x) dx = 1.25x^2 - 9.5x + C,$$

$$\int f_3(x) dx = 5 \ln x + C.$$

Команда	Аналитическое значение	Численный результат
<code>-test-integral f1 1 2 0.001</code>	$3 + 1.5 \ln \frac{3}{2} \approx 3.608198$	3.608469
<code>-test-integral f2 4 5 0.001</code>	1.750000	1.750000
<code>-test-integral f3 1 2 0.001</code>	$5 \ln 2 \approx 3.465736$	3.466041

Таблица 5: Тестирование функции `integral`

Полученные численные значения отличаются от аналитических на величину, соответствующую заданной точности $\varepsilon = 0.001$. Также была проверена работа режимов `-help`, `-roots` и `-iterations`. Ошибок выполнения при тестировании обнаружено не было.

Программа на Си и на Ассемблере

Исходные тексты программы имеются в архиве, приложенном к данному отчету. Архив содержит следующие файлы:

- `main.c` — основной код программы на языке Си;
- `funcs.asm` — реализация функций и производных на NASM32;
- `Makefile` — файл автоматической сборки.

Программа успешно собирается командой `make` и запускается командой `./program`. При запуске без аргументов программа выводит результаты вычислений для трех значений точности: 10^{-3} , 10^{-5} и 10^{-7} . При запуске с ключами командной строки доступны дополнительные режимы проверки и вывода результатов.

Анализ допущенных ошибок

В ходе выполнения задания программа была реализована в соответствии с требованиями варианта: функции и их производные вынесены в ассемблерный модуль, а основные численные алгоритмы, обработка ключей командной строки и вывод результатов реализованы на языке Си.

Во время итогового тестирования ошибок в работе программы обнаружено не было. Сборка проекта выполняется командой **make**, запуск без аргументов выводит результаты для трех значений точности, а дополнительные режимы **-roots**, **-iterations**, **-test-root** и **-test-integral** работают корректно. Полученные значения корней и интегралов согласуются с аналитическими проверками, а значение площади стабилизируется при уменьшении параметра точности ε .

Список литературы

- [1] Столяров А. В. Программирование на языке ассемблера NASM для ОС UNIX.
- [2] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 —М.: Изд-во МГУ, 1985.